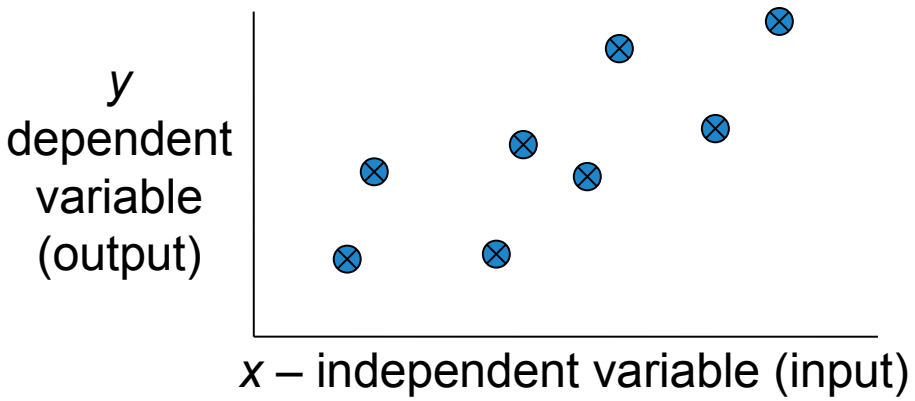
# REGRESSION

Sanjay Ranka
Distinguished Professor
Department of Computer and Information Science and Engineering
www.sanjayranka.com
sanjayranka@gmail.com
352 514 4213

All of the slides in this lecture are adapted from CS270 course at BYU

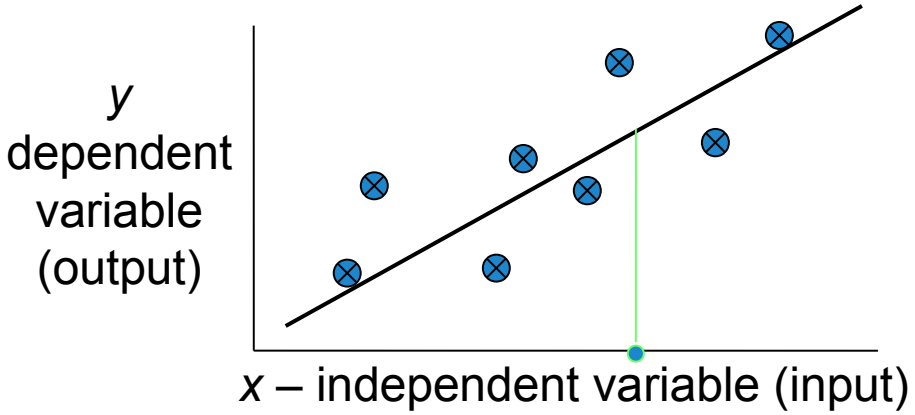
# Regression

- For classification, the output(s) is nominal
- In regression the output is continuous
  - Function Approximation
- Many models could be used – Simplest is linear regression
  - Fit data with the best hyper-plane which "goes through" the points



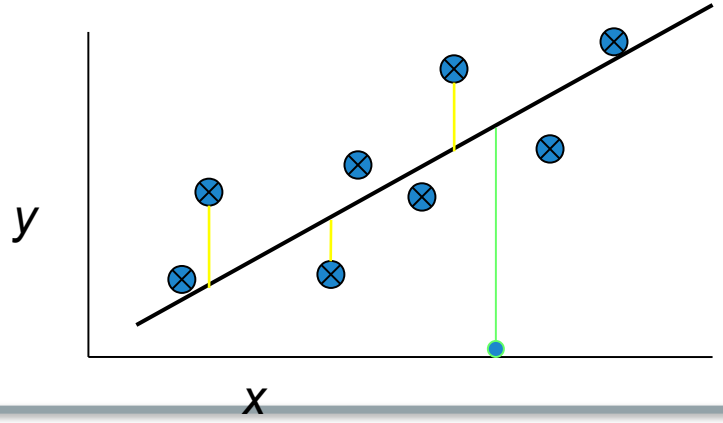
# Regression

- For classification, the output(s) is nominal
- In regression the output is continuous
  - Function Approximation
- Many models could be used – Simplest is linear regression
  - Fit data with the best hyper-plane which "goes through" the points



# Regression

- For classification the output(s) is nominal
- In regression the output is continuous
  - Function Approximation
- Many models could be used – Simplest is linear regression
  - Fit data with the best hyper-plane which "goes through" the points
  - For each point the difference between the predicted point and the actual observation is the residue



# Simple Linear Regression

- For now, assume just one (input) independent variable *x*, and one (output) dependent variable *y*
  - Multiple linear regression assumes an input vector **x**
  - Multivariate linear regression assumes an output vector **y**
- We "fit" the points with a line (i.e. hyperplane)
- Which line should we use?
  - Choose an objective function
  - For simple linear regression we use sum squared error (SSE)
    - $\Sigma\ (predicted_i - actual_i)^2 = \Sigma\ (residue_i)^2$
  - Thus, find the line which minimizes the sum of the squared residues (e.g. least squares)
  - This exactly mimics the case assuming data points were sampled from an actual target hyperplane with Gaussian noise added

# How do we "learn" parameters

- For the 2-*d* problem (line) there are coefficients for the bias and the independent variable (*y*-intercept and slope)

$$Y = \beta_0 + \beta_1 X$$

- To find the values for the coefficients (weights) which minimize the objective function we can take the partial derivates of the objective function (SSE) with respect to the coefficients. Set these to 0, and solve.

$$\beta_1 = \frac{n\sum xy - \sum x \sum y}{n\sum x^2 - \left(\sum x\right)^2}$$

$$\beta_0 = \frac{\sum y - \beta_1 \sum x}{n}$$

# Multiple Linear Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n$$

- There is a closed form for finding multiple linear regression weights which requires matrix inversion, etc.
- There are also iterative techniques to find weights
- One is the delta rule. For regression we use an output node which is not thresholded (just does a linear sum) and iteratively apply the delta rule – *For regression net is the output*

$$\Delta w_i = c(t - net)x_i$$

- Where $c$ is the learning rate and $x_i$ is the input for that weight
- Delta rule will update until minimizing the SSE, thus solving multiple linear regression
- There are other regression approaches that give different results by trying to better handle outliers and other statistical anomalies

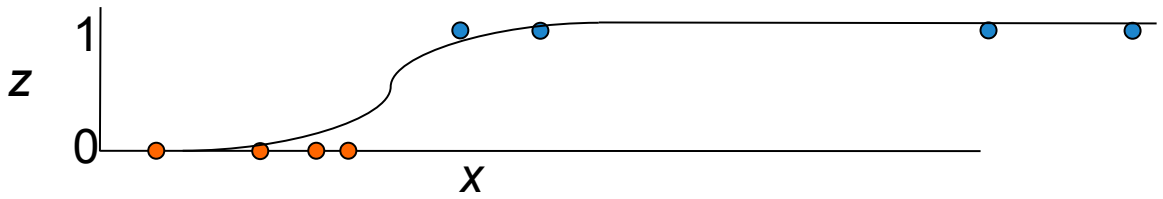
# SSE and Linear Regression

- SSE squares the difference of the predicted vs actual
- Don't want residues to cancel each other
- Could use absolute or other distances to solve problem

  $\Sigma\ |predicted_i - actual_i|$ :  L1 vs L2
- SSE leads to a parabolic error surface which is great for gradient descent
- Which line would least squares choose?
  - There is *always one* “best” fit with SSE (L2)
  - An L1 error can have multiple best fits

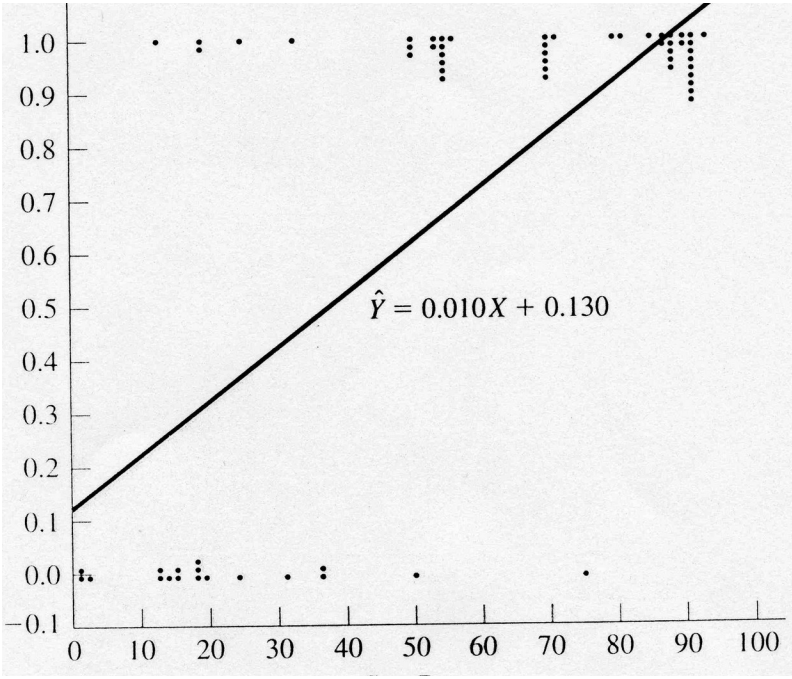
# Logistic Regression

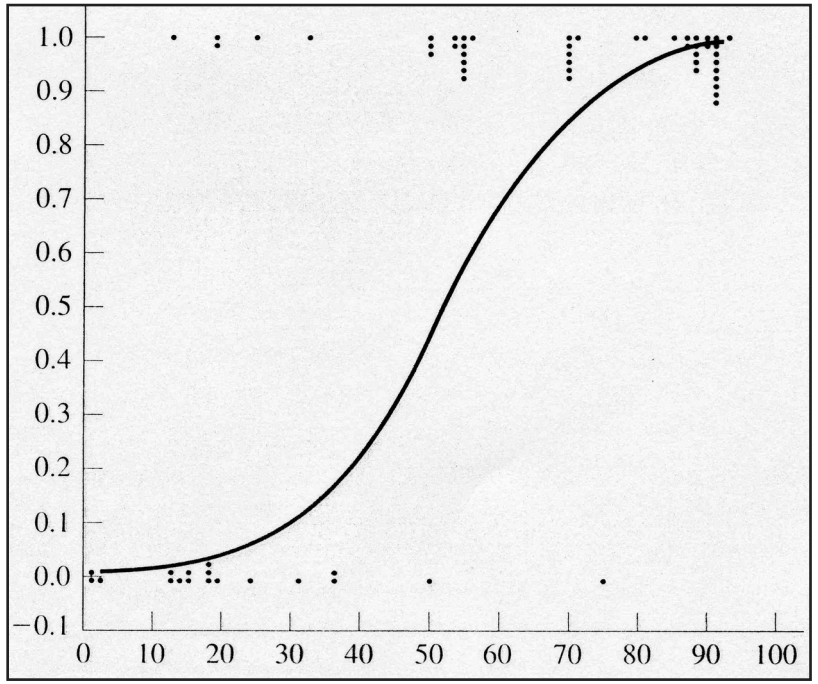
- One commonly used algorithm is Logistic Regression
- Assumes that the dependent (output) variable is binary which is often the case in medical and other studies. (Does person have disease or not, survive or not, accepted or not, etc.)
- Logistic Regression does a particular non-linear transform on the data after which it just does linear regression on the transformed data
- Logistic regression fits the data with a sigmoidal/logistic curve rather than a line and outputs an approximation of the probability of the output given the input



# Logistic Regression Example

- Age (X axis, input variable) – Data is fictional
- Heart Failure (Y axis, 1 or 0, output variable)
- If use value of regression line as a probability approximation
  - Extrapolates outside 0-1 and not as good empirically
- Sigmoidal curve to the right gives empirically good probability approximation and is bounded between 0 and 1



# Logistic Regression Approach

Learning

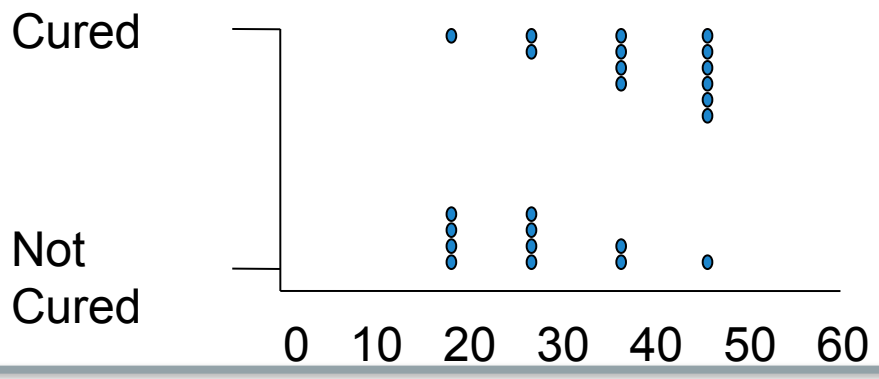
1. Transform initial input probabilities into log odds (logit)
2. Do a standard linear regression using the logit values
   - This effectively fits a logistic curve to the data, while still just doing a linear regression with the transformed input (ala quadric machine, etc.)

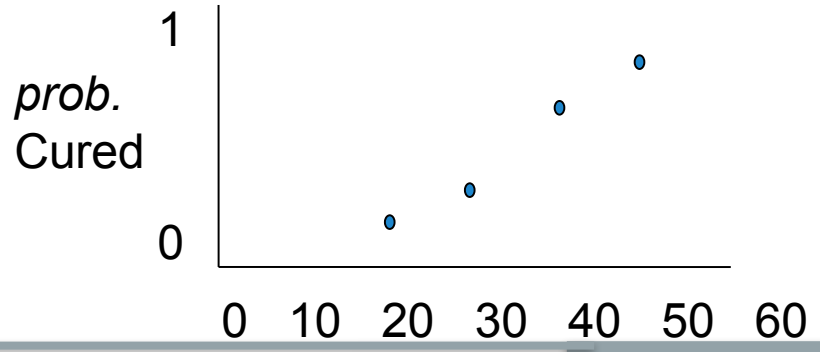
Generalization

3. Find the value for the new input on the logit line
4. Transform that logit value back into a probability

# Non-Linear Pre-Process to Logit (Log Odds)

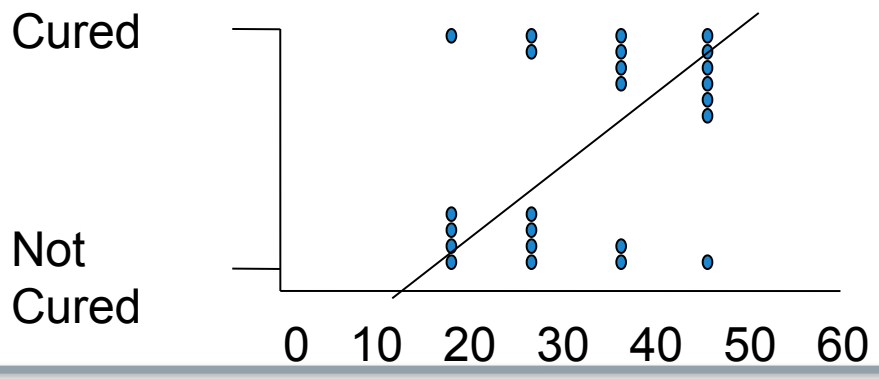
| Medication Dosage | # Cured | Total Patients | Probability: # Cured/Total Patients |
|---|---|---|---|
| 20 | 1 | 5 | .20 |
| 30 | 2 | 6 | .33 |
| 40 | 4 | 6 | .67 |
| 50 | 6 | 7 | .86 |

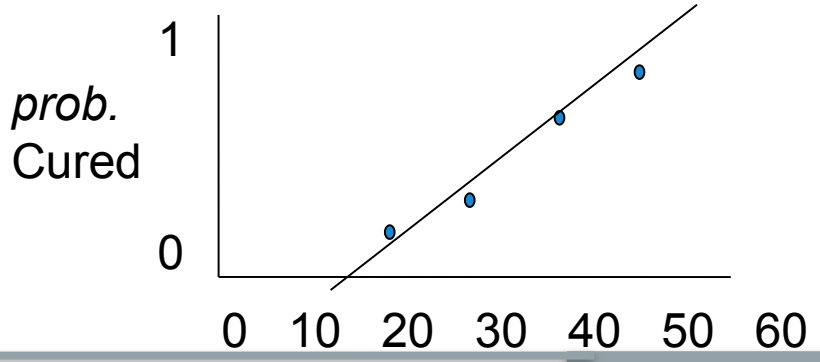


# Non-Linear Pre-Process to Logit (Log Odds)

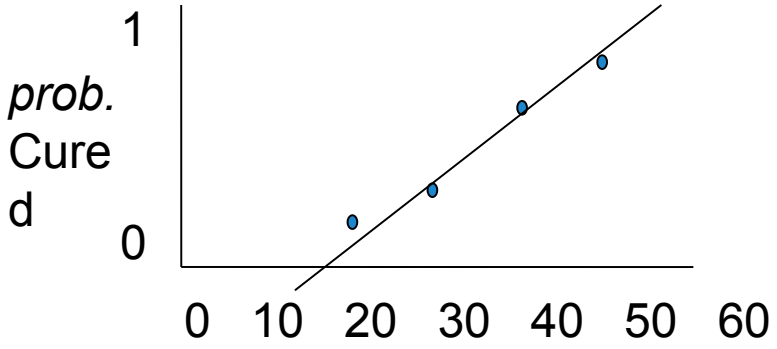
| Medication Dosage | # Cured | Total Patients | Probability: # Cured/Total Patients |
|---|---|---|---|
| 20 | 1 | 5 | .20 |
| 30 | 2 | 6 | .33 |
| 40 | 4 | 6 | .67 |
| 50 | 6 | 7 | .86 |

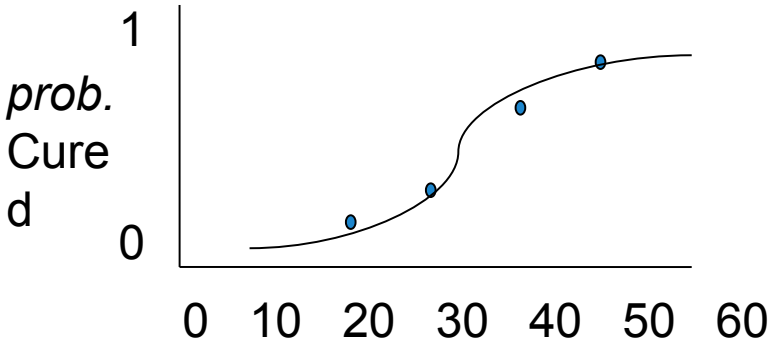


# Logistic Regression Approach

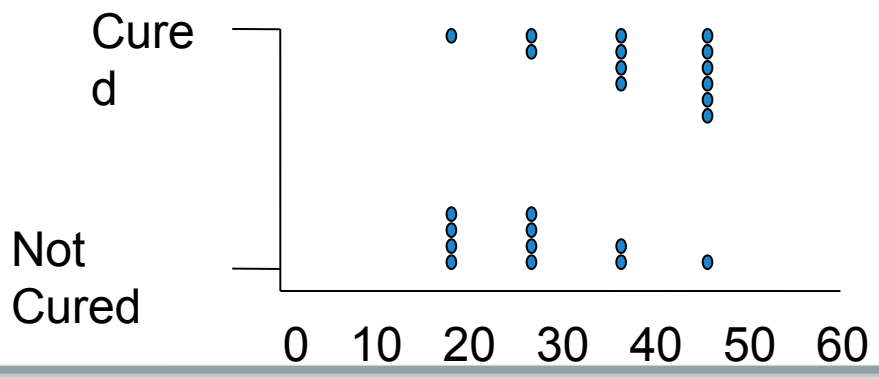
- Could use linear regression with the probability points, but that would not extrapolate well
- Logistic version is better but how do we get it?
- We do a non-linear pre-process of the input and then do linear regression on the transformed values – do a linear regression on the log odds - Logit

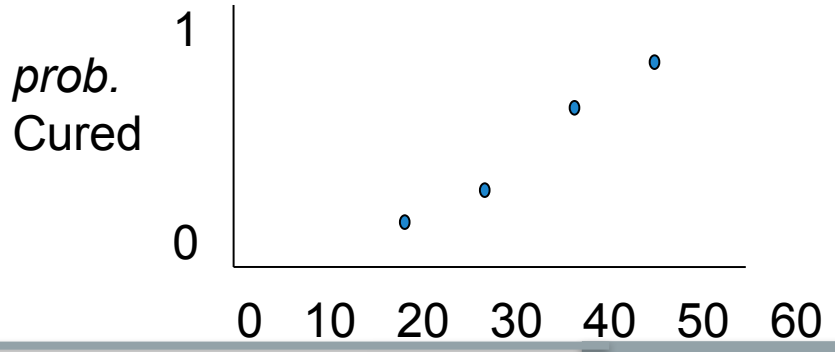


# Non-Linear Pre-Process to Logit (Log Odds)

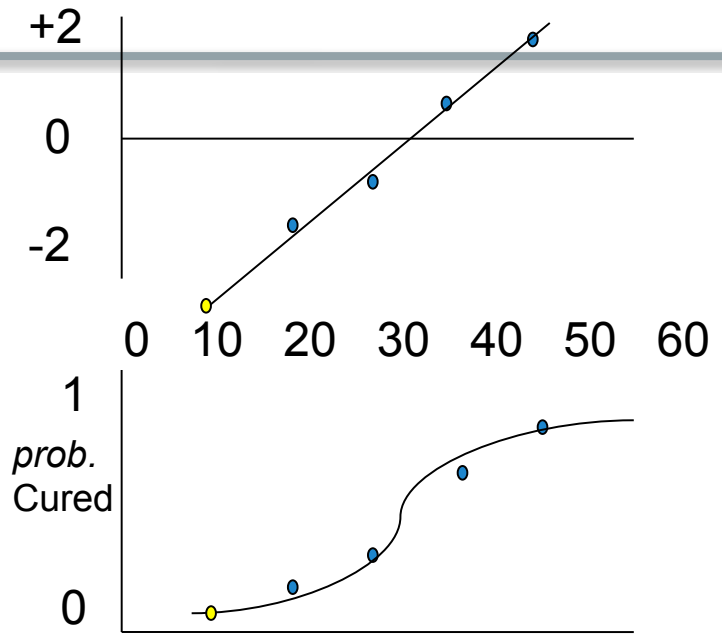
| Medication Dosage | # Cured | Total Patients | Probability: # Cured/Total Patients | Odds: *p*/(1-*p*) = # cured/ # not cured | Logit Log Odds: ln(Odds) |
|---|---|---|---|---|---|
| 20 | 1 | 5 | .20 | .25 | -1.39 |
| 30 | 2 | 6 | .33 | .50 | -0.69 |
| 40 | 4 | 6 | .67 | 2.0 | 0.69 |
| 50 | 6 | 7 | .86 | 6.0 | 1.79 |



# Regression of Log Odds

| Medication Dosage | # Cured | Total Patients | Probability: # Cured/Total Patients | Odds: $p/(1-p)$ = # cured/ # not cured | Log Odds: ln(Odds) |
|---|---|---|---|---|---|
| 20 | 1 | 5 | .20 | .25 | -1.39 |
| 30 | 2 | 6 | .33 | .50 | -0.69 |
| 40 | 4 | 6 | .67 | 2.0 | 0.69 |
| 50 | 6 | 7 | .86 | 6.0 | 1.79 |




- $y = .11x - 3.8$ - Logit regression equation
- Now we have a regression line for log odds (logit)
- To generalize, we use the log odds value for the new data point
- Then we transform that log odds point to a probability: $p = e^{\text{logit}(x)}/(1+e^{\text{logit}(x)})$
- For example assume we want $p$ for dosage = 10

$$\text{Logit}(10) = .11(10) - 3.8 = -2.7$$

$p(10) = e^{-2.7}/(1+e^{-2.7}) = .06$ [note that we just work backwards from logit to $p$]

- These $p$ values make up the sigmoidal regression curve (which we never have to actually plot)

# Non-Linear Regression

- Note that linear regression is to regression what the perceptron is to classification
  - Simple, useful models which will often underfit
- The more powerful classification models which we will be discussing going forward in class can usually also be used for non-linear regression
  - MLP with Backpropagation, Decision Trees, Nearest Neighbor, etc.
- They can learn functions with arbitrarily complex high dimensional shapes

# Summary

- Linear Regression and Logistic Regression are nice tools for many simple situations
  - But both force us to fit the data with one shape (line or sigmoid) which will often underfit
- Intelligible results
- When problem includes more arbitrary non-linearity then we need more powerful models which we will introduce
  - Yet non-linear data transformations (e.g. Quadric perceptron) can help in these cases while still using a linear model for learning
- These models are commonly used in data mining applications and also as a "first attempt" at understanding data trends, indicators, etc.